LeadFarm v0.1.0

Production Readiness Audit Report

Project	LeadFarm — Precision Agriculture Platform
Stack	Next.js 16 + React 19 + Supabase + Tailwind CSS 4
Auditor	Senior Full-Stack Engineer (Automated)
Date	March 28, 2026
Scope	Database, Frontend, API, Security, Code Quality

1. Executive Summary

This report presents the findings of a comprehensive production readiness audit of the LeadFarm platform. The audit covered database schema and security, frontend code quality, CRUD operations, error handling, and API endpoints. **7 critical issues were found and fixed during this audit.** Several warnings remain that should be addressed before a public-facing production launch.

OVERALL VERDICT	CONDITIONAL PASS

The app is functional and the fixed CRUD pipeline now works correctly with the database. However, the RLS security policies must be tightened before exposing the app to the public internet.

2. Critical Issues Found and Fixed

During this audit, 7 critical bugs were identified and fixed in the codebase. These would have caused all stock movements to fail when saving to the database.

Area	Status	Detail
DB Enum Mismatch	**PASS**	The category field was sending movement categories (e.g. 'entree_fournisseur') instead of product_category enum values (e.g. 'fongicide'). Fixed in all 4 modals.
Invalid movement_type	**PASS**	Code sent 'adjustment' and 'treatment_consumption' but the DB only accepts: entree, sortie, transfert, retour. Fixed Zod schema and all modals.
Culture Enum	**PASS**	UI sent uppercase display values ('A PEPINS') but DB expects culture_type enum ('a_pepins'). Added mapping dictionary.
Status Enum	**PASS**	Zod schema had 'over' but DB enum is 'overstock'. Fixed in validations.ts.
Silent Error Swallow	**PASS**	3 modals (Consommation, Ajustement, Controle) had empty catch blocks. Added error state and red error banner display to all.
Stale Closure	**PASS**	handleSubmit in NewEntryModal was missing 'products' in useCallback deps, causing stale data when looking up product category. Fixed.
Error Serialization	**PASS**	Supabase PostgrestError has non-enumerable properties. catch block now explicitly extracts .message, .code, .details, .hint for display.

3. Database Audit

3.1 Schema Overview

The Supabase PostgreSQL database contains 27 tables across the public schema, covering products, movements, stock levels, treatments, parcelles, operators, alerts, and supporting entities (regions, zones, sites, profiles, orgs).

3.2 Indexes

Good index coverage on frequently queried columns: movements has 6 indexes (date, product_id, category, culture, site_name, movement_type). Products indexed on trade_name and category. Stock_levels has a unique constraint on product_id.

3.3 Foreign Keys

The movements table has proper FK constraints to products, suppliers (via supplier_id and distributor_id), and sites. This ensures referential integrity at the database level.

3.4 Row Level Security (RLS)

Area	Status	Detail
RLS Enabled	**PASS**	RLS is ON for all 27 public tables.
Movements Policy	**FAIL**	Policy is 'Allow all for anon' with qual=true. Any anonymous user can read/write/delete all movements. Must be scoped to authenticated users with org-based filtering.
Products Policy	**FAIL**	Same 'Allow all for anon' — full public access. Needs auth + org scoping.
Stock Levels Policy	**FAIL**	Same 'Allow all for anon'. Must restrict to org members.
Suppliers Policy	**FAIL**	Same permissive policy. Supplier data is business-sensitive.
Parcelles Policy	**PASS**	Correctly scoped: read/write filtered by org_id = ANY(auth_org_ids()). Good pattern.
Treatments Policy	**FAIL**	Same 'Allow all for anon'. Treatment data needs protection.

Recommendation: Replicate the parcelles RLS pattern (org_id-based scoping with auth_org_ids()) across all tables before production. The current 'Allow all for anon' policies mean anyone with the Supabase anon key can read, modify, or delete all data.

4. Frontend Code Audit

4.1 Architecture

The app uses Next.js 16 App Router with 13 page routes, a data-provider abstraction layer with Supabase/mock-data fallback, and Zod validation for all mutations. The architecture is clean and well-organized with proper separation of concerns.

Area	Status	Detail
Page Routes	PASS	13 routes all properly structured under src/app/ with correct 'use client' directives.
Data Layer	PASS	data-provider.ts provides clean abstraction with snake_case mapping. Fallback to mock-data when Supabase not configured.
Zod Validation	PASS	All mutation functions validate input before DB writes. Schemas now match DB enums.
Error Handling	PASS	All 4 stock modals now surface errors to users with styled red banners (previously silent).
TypeScript	WARN	Some 'any' types used in data-provider mappers. Not a blocker but reduces type safety.
Bundle Size	WARN	mock-data.ts is 50KB of hardcoded data that ships to production. Should be lazy-loaded or removed when Supabase is configured.

4.2 Stock Modals (CRUD Pipeline)

Area	Status	Detail
NewEntryModal	PASS	Full form with product search, auto-category, locked field, supplier tracking. All enum values now correctly mapped to DB.
ConsommationModal	PASS	Multi-step wizard with product lines. movement_type fixed from 'treatment_consumption' to 'sortie'. Error feedback now displayed.
AjustementModal	PASS	Inventory adjustment with ecart display. movement_type fixed from 'adjustment' to 'entree/sortie'. Error feedback added.
ControleModal	PASS	Quality control checklist with conformity verdict. movement_type fixed. Error feedback added.
InventoryModal	WARN	Validates physical counts but error feedback for updateStockLevel failures is only in console. Consider adding user-visible error.

5. API Routes Audit

Area	Status	Detail
GET /api/v1/*	PASS	All GET routes have proper query parameter handling and error responses.
POST /api/v1/movements	FAIL	No input validation — body is inserted directly into Supabase. Must add Zod validation (movementSchema.parse) before insert.
POST /api/v1/products	WARN	Should validate with productSchema before insert. Currently trusts client input.
Rate Limiting	FAIL	No rate limiting on any API route. Vulnerable to abuse.
Auth Middleware	WARN	middleware.ts exists but API routes don't verify auth tokens. Combined with open RLS policies, this is a security gap.

6. Remaining Warnings (Non-Blocking)

Area	Status	Detail
teneurMAUnit field	WARN	Used in stock-cards.tsx but the column does not exist in the products DB table. Only available in mock data. Displays empty string gracefully but won't show units from Supabase.
useData.ts Hook	WARN	The eslint-disable for exhaustive-deps on the base useAsyncData hook masks a potential infinite re-render loop. Should refactor deps handling.
PDF Export	WARN	The 'Export PDF' button in InventoryModal only calls window.print(). Not a real PDF export. Consider using reportlab or html2pdf.
Accessibility	WARN	Modal dropdowns lack ARIA roles (listbox, option). Close buttons missing aria-label. Keyboard navigation within custom dropdowns is incomplete.
Operator Field	WARN	ConsommationModal captures 'operateur' input but never includes it in the insertMovement call. The value is lost on save.
Duplicate Data Layer	WARN	Two hook files exist: useData.ts (used by pages) and useSupabase.ts (unused). Consider consolidating to avoid confusion.

7. Production Launch Checklist

Priority	Action Item
MUST DO	Fix RLS policies — replicate parcelles pattern (org_id scoping) to all tables
MUST DO	Add Zod validation to all POST API routes
MUST DO	Add authentication checks to API routes (verify Supabase JWT)
SHOULD DO	Add rate limiting middleware (e.g. next-rate-limit or Vercel edge config)
SHOULD DO	Remove or lazy-load mock-data.ts in production builds
SHOULD DO	Add ARIA roles to custom dropdown components for accessibility
NICE TO HAVE	Add the 'operateur' field to the movements table and include in inserts
NICE TO HAVE	Add teneur_ma_unit column to products table
NICE TO HAVE	Implement real PDF export using reportlab or client-side html2pdf
NICE TO HAVE	Consolidate useData.ts and useSupabase.ts into a single hooks file

Summary: The 7 critical bugs that blocked stock movement CRUD have been fixed. The application is functional for internal/team use. Before public production launch, the 3 'MUST DO' items (RLS policies, API validation, auth checks) must be completed to prevent unauthorized data access.